ENGINEERING PRD • LAB MILESTONE

Status: Approved Baseline
Target Year: Academic Cycle

Type-Inferring Source-to-Source Transpiler

Scope: Mini-Python to Strict TypeScript • **Evaluation:** Compiler Design Reviews 1–3

1. PRODUCT VISION & PROBLEM STATEMENT

Dynamic scripting languages like Python enable rapid, friction-free prototyping but lack deterministic compile-time type guarantees. Conversely, strongly typed platforms like TypeScript enforce structural integrity and eliminate type-directed runtime errors in large-scale systems.

Core Objective: Develop an end-to-end source-to-source compiler (transpiler) that ingests untyped Mini-Python, resolves indentation structures via off-side lexical tracking, applies local static constraint-based type inference, and emits idiomatic, strictly typed TypeScript that strictly avoids the `any` escape hatch.

2. LANGUAGE FEATURE SCOPE & TARGET SEMANTIC MAPPING

FEATURE CATEGORY	MINI-PYTHON SYNTAX (INPUT)	STRICT TYPESCRIPT (OUTPUT)	INFERRED TYPE & SEMANTIC RULE

Primitive Bindings	`count = 10`
	`name = "Alice"`
	`is_valid = True`
		`let count: number = 10;`
		`let name: string = "Alice";`
		`let isValid: boolean = true;`
			Direct inference from primitive literals (`number`, `string`, `boolean`). Translates identifiers to idiomatic camelCase.

Arithmetic & Expressions	`total = 5 + 3 * 2` `ratio = total / 2`	`let total: number = 5 + 3 * 2;` `let ratio: number = total / 2;`	Recursive evaluation of binary operator AST subtrees. Type mismatches (e.g. arithmetic on strings) throw compile-time semantic diagnostics.

Conditionals & Control	`if total > 10:`
	`  print("High")`
	`else:`
	`  print("Low")`
		`if (total > 10) {`
		`  console.log("High");`
		`} else {`
		`  console.log("Low");`
		`}`
			Evaluates guard expression to boolean; maps `print` to `console.log`; synthesizes block boundaries (`{ }`).

While Loops	`while count > 0:`
	`  count = count - 1`
		`while (count > 0) {`
		`  count = count - 1;`
		`}`
			Validates boolean termination conditions and guarantees that mutable variable mutations maintain invariant static types.

Functions & Calls	`def add(a, b):` `  return a + b`	`function add(a: number, b: number): number {` `  return a +`	Bidirectional constraint propagation: derives parameter types across AST call sites and unifies return statement expression trees.

3. SYSTEM ARCHITECTURE PIPELINE



1. Lexical Analyzer & Off-Side Stack
Evaluates column indentation per line, emitting virtual `INDENT` and `DEDENT` tokens to normalize block nesting for syntax parsing.

2. Recursive Descent AST Parser
Generates a structured Abstract Syntax Tree with line/column diagnostic anchors, preventing ambiguity across statements and operator precedence.

3. Semantic & Local Type Inference
Maintains scoped symbol tables, propagates call-site parameter constraints, and unifies return branches while rejecting conflicting reassignments.

4. TypeScript Code Generation
Traverses decorated AST via Visitor Pattern, injecting explicit type signatures (`: number`, `: string`), block braces, and semicolons.

4. TECHNICAL STACK & IMPLEMENTATION DETAILS

- **Language & Runtime:** TypeScript on Node.js. Rigid AST node typings eliminate class casting runtime bugs.
- **Architecture Pattern:** Decoupled Visitor Pattern traversing AST representations for analysis and emission.
- **Quality Assurance:** Vitest suite executing transpile-and-compile pipelines, verifying zero errors under strict `tsc`.
- **CLI Distribution:** Standard invocation via POSIX arguments: `node transpile.js <input.py> -o <output.ts>`.

5. REVIEW MILESTONE DELIVERABLES & RUBRIC MAPPING

MILESTONE & SCHEDULE	EVALUATION RUBRIC TARGETS	SCOPE OF DELIVERABLES & ARTIFACTS
Review 1 (1st Week of October)	• Problem ID & Objectives (2M) • Literature Survey (2M) • Novelty Formulation (2M) • 25% Implementation (2M) • Presentation & Q/A (2M)	Finalize PRD and algorithmic survey on local type inference. Working Lexer with virtual indentation stack (`INDENT` / `DEDENT`). Functional Recursive Descent Parser emitting base AST for bindings, binary math, and conditionals.
Review 2 (3rd Week of October)	• 75% Implementation (6M) • Progress Documentation (2M) • Presentation & Q/A (2M)	Scope-aware Symbol Table and Type Inference engine for primitives and function signatures. Grammar and AST support for `while` loops, function blocks, and array constructs. Submission of interim technical report documenting typing rules.
Review 3 (4th Week of October)	• 100% System Demo (3M) • Individual Contribution (3M) • Research Manuscript / Patent (4M)	End-to-end Code Generator with automated CLI tool verified via `tsc`. Comprehensive integration test suite covering type-error rejections. Final IEEE-format manuscript: *"Static Type Inference and Code Generation for Python-to-TypeScript Source-to-Source Compilation"*.